Power and Sample Size Planning

PSYC 201A, Simulation Lab 4

Hew Yeh

2026-11-03

Table of contents

Tips before getting started 2

Setup **2**
Import the libraries 2

Part 1: Probing your power intuitions **3**

Part 2: Same answer, two ways **5**

Part 3: Why you shouldn't power on the published effect **7**

Part 4: Why you shouldn't power on your pilot **11**

Part 5: Small telescopes, or how big should a replication be? **13**

Part 6: When your sample is fixed, flip the question **15**

Part 7: More people or more trials? **17**

Part 8: Getting an effect size **18**
8a. From raw data: Cohen's d by hand 18
8b. Within-subjects designs: which d? 20
8c. From the numbers a paper reports 21
8d. How sure are you about that effect size? 22

Part 9: Your project **24**

Bonus: mixed models **28**
Doing the same thing with `simr` 29

References 30

Tips before getting started

This document accompanies the sampling and sample size planning practicum in Psych 201a (Experimentology, Ch. 10). In Lab 2 you built a power curve when you got to choose the true effect. Real planning is harder, because nobody tells you the true effect. Here you will see what goes wrong when you plan from a published effect or a pilot, what the alternatives are, and then write the sample size justification for your own project.

Open this file by double-clicking `power_simulation_lab_4.Rproj` first, then opening it from the Files pane. You need the tidyverse, which you already have, and one new package, `pwr`.

To understand what a function does, type `?function_name` in the Console (e.g., `?pwr.t.test`).

`ch10_power_simulations_solutions.qmd` is the answer key. Try each part first.

`error: true` is set at the top of this file, so it still renders while some chunks are unfinished: those chunks show their error in red. Each part starts with its own `set.seed()`. Later parts use the functions defined in Parts 2 and 3, so run those first.

Short on time? Do Parts 1, 3, 6, and 9, plus Part 8 if your paper doesn't report an effect size. Run the chunks in Part 2 that define the functions first, because Parts 3 and 8 use them. Two of the options in Part 9 build on parts you'd skip: safeguard power uses `d_ci()` from Part 8d (run that chunk), and the 2.5× rule comes from Part 5. Part 9 is the one you hand in.

Setup

```
set.seed(2026) # same "random" numbers on every render
```

Import the libraries

```
options(repos = c(CRAN = "https://cloud.r-project.org"))
install.packages("pwr") # run once
install.packages("tidyverse")
```

```r
library(tidyverse)
library(pwr)

theme_set(theme_classic(base_size = 14))
```

Part 1: Probing your power intuitions

A few terms first. **Power** is the chance that a study gets a significant result when the effect is real. **Cohen's d** measures how big the difference between two groups is: the difference between the group means divided by the standard deviation within a group. So d = .5 means the groups differ by half a standard deviation. As a rough guide, d = .2 is small, .5 is medium, and .8 is large. **r** is a correlation, from −1 to 1. **alpha**, called `sig.level` in `pwr`, is the p-value cutoff for calling a result significant, here .05. **Two-sided** means an effect in either direction counts.

Before you touch any code, guess the power of each of these studies. Type your guesses into the table. Be honest: this works better if you are wrong.

Test	Effect	Total N	Your guess	Power from `pwr`
Two-group t-test	d = .5	24	15%	22%
Two-group t-test	d = .8	36	25%	65%
Two-group t-test	d = .3	120	15%	37%
Correlation	r = .5	36	25%	59%
Correlation	r = .3	80	20%	48%

Now check them. **Watch out:** in `pwr.t.test()`, `n` means participants *per group*, not the total. This is the most common way to get a power analysis wrong, in `pwr` and in G*Power. In `pwr.r.test()`, `n` is just the number of people.

To do: Fill in each `___`, then fill in the last column of the table.

```r
pwr.t.test(n = 12, d = 0.5, sig.level = .05)$power
```

```
[1] 0.2161431
```

```r
pwr.t.test(n = 18, d = .8, sig.level = .05)$power
```

```
[1] 0.6450047
```

```r
pwr.t.test(n = 60, d = .3, sig.level = .05)$power
```

```
[1] 0.3708022
```

```r
pwr.r.test(n = 18, r = 0.5, sig.level = .05)$power
```

```
[1] 0.5860955
```

```r
pwr.r.test(n = 40, r = .3, sig.level = .05)$power
```

```
[1] 0.4772542
```

Power depends on four numbers: `n`, the effect size, `sig.level`, and `power`. Give `pwr` any three and it solves for the fourth. This is the classic **a priori power analysis**, done before you collect any data: leave `n` out, and set `power`.

To do: How many participants, in total, do you need for 80% power to detect d = .5? What about d = .3? Round up, because you can't run part of a person, and remember that `n` is per group.

Hew's Answer: It seems that we would need 128 people for a $d = .5$, and 352 people for $d = .3$.

```r
n_d5 <- pwr.t.test(d = 0.5, power = .8, sig.level = .05)$n
n_d3 <- pwr.t.test(d = 0.3, power = .8, sig.level = .05)$n

ceiling(n_d5) * 2 # total N
```

```
[1] 128
```

```r
ceiling(n_d3) * 2
```

```
[1] 352
```

Part 2: Same answer, two ways

G*Power, `pwr`, and simulation all answer the same question. Formulas are instant, but they exist only for standard tests. Simulation is slower, but it works for any design you can write down as code. When a formula exists, the two should agree, and that is a useful check on your simulation.

Here is the machinery from Lab 2, already written for you.

```
make_tea_data <- function(n_total = 48,
                          sigma = 1.25, # sd of ratings within a condition
                          delta = 1.5) { # difference between conditions
  n_half <- n_total / 2
  tibble(condition = c(rep("milk first", n_half), rep("tea first", n_half)),
         rating = c(round(rnorm(n_half, mean = 3.5 + delta, sd = sigma)),
                    round(rnorm(n_half, mean = 3.5, sd = sigma)))) |>
    mutate(rating = pmin(pmax(rating, 1), 10)) # keep ratings on the 1-10 scale
}

run_one_experiment <- function(n_total, delta) {
  this_experiment <- make_tea_data(n_total = n_total, delta = delta)
  t.test(rating ~ condition, data = this_experiment, var.equal = TRUE)$p.value
}

estimate_power <- function(n_total, delta, n_sims = 500) {
  mean(map_dbl(1:n_sims, \(i) run_one_experiment(n_total, delta)) < .05)
}
```

`pwr` needs Cohen's d, so divide the difference in scale points by the standard deviation: `delta / sigma`. With `delta = 1` and `sigma = 1.25`, d = .8.

To do: Fill in the blanks to compute power at each sample size both ways, using `delta = 1`. Careful: inside `\(n) ...`, `n` is the *total* number of participants, but `pwr.t.test()` wants `n` per group. This runs 5,000 simulated experiments, so let it finish.

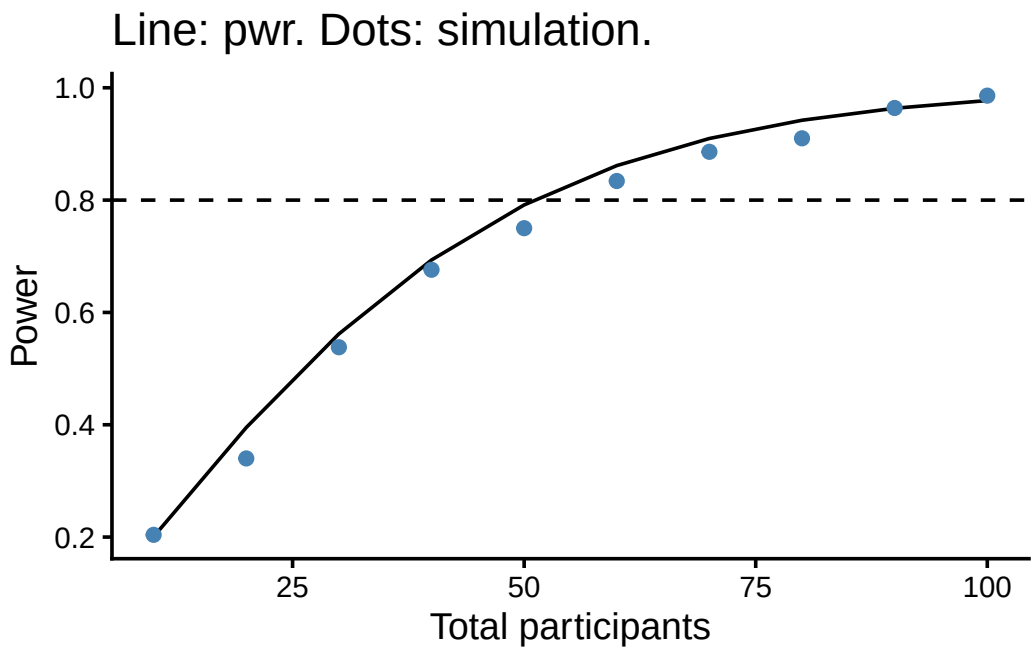
```
set.seed(2)

power_check <- tibble(n_total = seq(10, 100, by = 10)) |>
  mutate(simulated = map_dbl(n_total, \(n) estimate_power(n, delta = 1)),
         formula = map_dbl(n_total, \(n) pwr.t.test(n = n / 2, d = 0.8)$power))

power_check
```

```
# A tibble: 10 x 3
   n_total simulated formula
     <dbl>     <dbl>   <dbl>
 1      10     0.204   0.201
 2      20     0.34    0.395
 3      30     0.538   0.562
 4      40     0.676   0.693
 5      50     0.75    0.791
 6      60     0.834   0.861
 7      70     0.886   0.910
 8      80     0.91    0.942
 9      90     0.964   0.964
10     100     0.986   0.977
```

```
power_check |>
  ggplot(aes(x = n_total)) +
  geom_line(aes(y = formula)) +
  geom_point(aes(y = simulated), color = "steelblue", size = 2) +
  geom_hline(yintercept = .8, linetype = "dashed") +
  labs(x = "Total participants", y = "Power",
       title = "Line: pwr. Dots: simulation.")
```



To do: Do they agree? If there is a gap, with the dots a little below the line, why might that

be? (Hint: look at what `make_tea_data()` does to the ratings. What does `pwr` assume about them?)

Hew's Answer: The points deviate a bit from the curve. This indicates that the computed power through a bunch of `pwr.t.test` functions will be slightly different than simulating/doing Monte Carlo. This is because the `pwr.t.test` function is using the closed form solution for a t-test rather than using a simulation approach which has subject to noise.

Part 3: Why you shouldn't power on the published effect

In Lab 2 you got to choose the true effect. The usual real-world move is to look up the original paper and plug in its effect size. Let's see what happens if you do that.

`simulate_study()` runs one two-group study with a true effect of `true_d`, and returns the observed effect size and p-value. The observed d comes from the t statistic, using a shortcut (Part 8a shows why it works): $d = t \times \sqrt{(2/n)}$, where n is the sample size per group.

```
simulate_study <- function(n_per_group, true_d) {
  treatment <- rnorm(n_per_group, mean = true_d, sd = 1)
  control <- rnorm(n_per_group, mean = 0, sd = 1)
  test <- t.test(treatment, control, var.equal = TRUE)
  tibble(d_hat = unname(test$statistic) * sqrt(2 / n_per_group),
         p = test$p.value)
}

simulate_study(n_per_group = 20, true_d = 0.3) # run this a few times
```

```
# A tibble: 1 x 2
  d_hat     p
  <dbl> <dbl>
1 0.504 0.120
```

Imagine a whole literature: 5,000 labs each run the same study with 20 people per group. The true effect is d = .3.

```
set.seed(3)

literature <- tibble(study = 1:5000) |>
  mutate(result = map(study, \(i) simulate_study(n_per_group = 20, true_d = 0.3))) |>
  unnest(result)
```

Journals mostly publish significant results in the predicted direction.

To do: Add a column `published` that is `TRUE` when `p < .05` and `d_hat > 0`. Then compute two means: `d_hat` across all studies, and `d_hat` across the published ones only.

```
literature <- literature |>
  mutate(published = p < .05 & d_hat > 0)

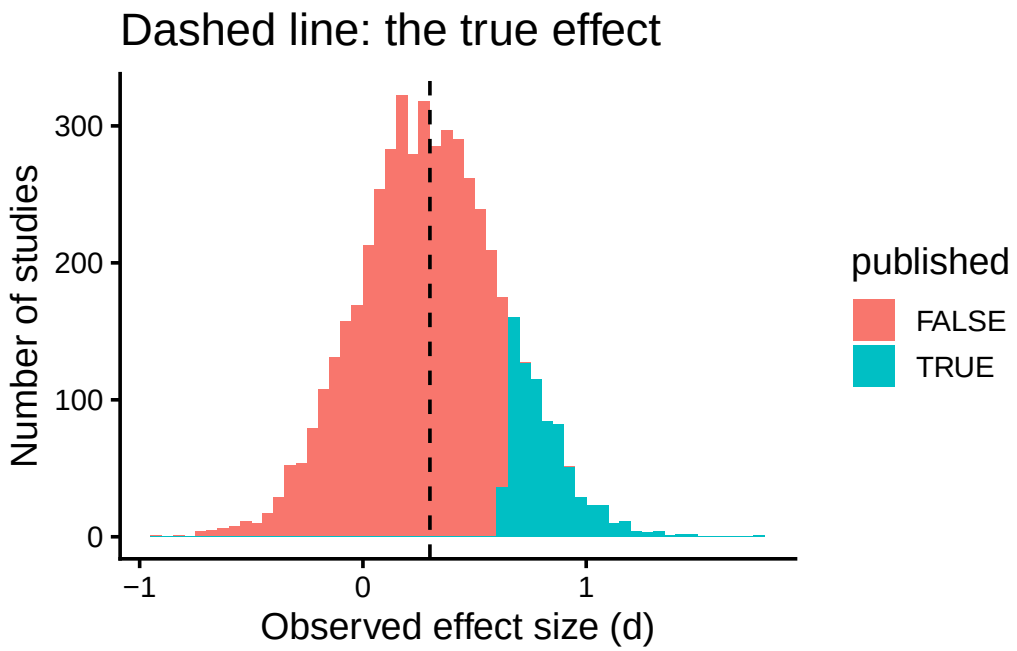
literature |>
  group_by(published) |>
  summarize(n_studies = n(),
            mean_d = mean(d_hat))
```

```
# A tibble: 2 x 3
  published n_studies mean_d
  <lgl>         <int>  <dbl>
1 FALSE          4232  0.214
2 TRUE            768  0.812
```

```
mean(literature$d_hat) # all studies
```

```
[1] 0.3058156
```

```
literature |>
  ggplot(aes(x = d_hat, fill = published)) +
  geom_histogram(binwidth = .05, boundary = 0) +
  geom_vline(xintercept = 0.3, linetype = "dashed") +
  labs(x = "Observed effect size (d)", y = "Number of studies",
       title = "Dashed line: the true effect")
```



To do: What percent of studies got published, and what does that tell you about the power of these studies? What is the average published effect, compared to the truth? (Hint: divide `n_studies` for the published row by 5,000, and compare it with `pwr.t.test(n = 20, d = 0.3)$power`.)

```
# n_studies are the amount of published studies
768/5000
```

```
[1] 0.1536
```

About 15.3% of the studies got published. The average published effect is $d = .8$ which is much larger than the $d = .3$.

Note that 15.3% is the same power that we get for $d = .3, n = 20$.

Now you read one of those published papers and plan a replication with 80% power, using its effect size. `n_for_80()` does the a priori calculation.

```
n_for_80 <- function(d) {
  ceiling(pwr.t.test(d = d, power = .8, sig.level = .05)$n)
}

n_for_80(0.8) # what you'd plan from a typical published study (per group)
```

```
[1] 26
```

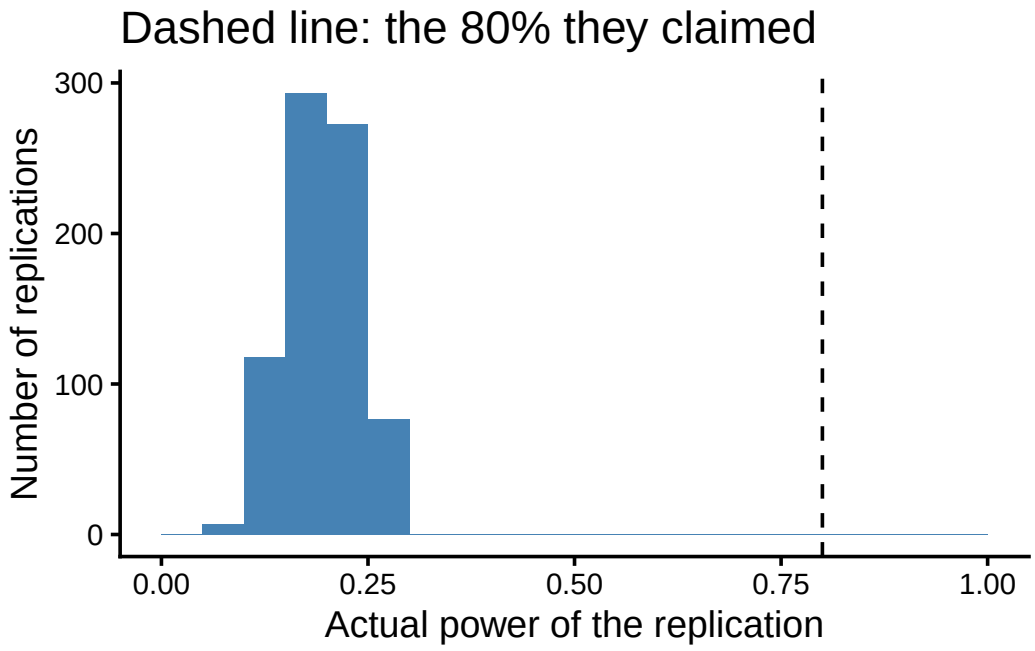
To do: For every published study, work out the planned sample size, then the power that replication *actually* has, given that the truth is d = .3. Fill in the blanks. This does about 750 power analyses, so let it finish.

```
replications <- literature |>
  filter(published) |>
  mutate(planned_n = map_dbl(d_hat, n_for_80),
         actual_power = map_dbl(planned_n, \(n) pwr.t.test(n = n, d = 0.3)$power))

mean(replications$actual_power)
```

```
[1] 0.1929818
```

```
replications |>
  ggplot(aes(x = actual_power)) +
  geom_histogram(binwidth = .05, boundary = 0, fill = "steelblue") +
  geom_vline(xintercept = .8, linetype = "dashed") +
  xlim(0, 1) +
  labs(x = "Actual power of the replication", y = "Number of replications",
       title = "Dashed line: the 80% they claimed")
```



To do: Answer these questions.

1. Everyone claimed 80% power. What did they really have, on average?
   They actually had 19.3% power .
2. How many participants per group would it have taken to have 80% power for the true effect? Compare that to `median(replications$planned_n)`.
   `n_for_80(0.3)` gives us 176. But the median was 27.5 (cannot have a fractional person so that would be roughly 26–28).
3. If one of these replications comes out non-significant, how much does that tell you?
   A non-significant finding from one of these replications would not tell me much because they were under powered to find the correct effect in the first place. It also cannot tell us if the original effect was correct… Which makes me wonder if part of the replication crisis is that the original studies were underpowered so replications with similar setups would be underpowered and unlikely to replicate the same effect size.

Part 4: Why you shouldn't power on your pilot

The next idea is usually "fine, I'll run a small pilot and use that effect size." There's no publication filter on your own pilot, so maybe that's better?

Say the true effect is d = .4, and you pilot 10 people per group.

To do: Write this part yourself, following Part 3.

1. Simulate 2,000 pilots with `simulate_study()` and call them `pilots`.
2. Add a column `go`: only continue if the pilot looks promising, `d_hat > 0.2`. Lots of labs do this informally. `go` will be `TRUE` or `FALSE` for each pilot.
3. What percent of pilots lead you to abandon a study of a real effect? (Hint: `mean()` of a `TRUE`/`FALSE` column gives the proportion that are `TRUE`, and `!` turns `TRUE` into `FALSE` and back.)
4. Keep only the pilots that continue, with `filter(go)`, and call the result `followups`. Then compute `planned_n` and `actual_power` as in Part 3 (the truth is now d = .4).
5. Plot a histogram of `planned_n`, and report the 10th, 50th, and 90th percentiles with `quantile()`. The histogram is very lopsided, so add `scale_x_log10()` to your plot.

```
set.seed(4)

pilots <- tibble(pilot = 1:2000) |>
  mutate(result = map(pilot, \(i) simulate_study(n_per_group = 10, true_d = 0.4))) |>
  unnest(result) |>
  mutate(go = d_hat > 0.2)

mean(!pilots$go) # share of pilots where you give up on a real effect
```

```
[1] 0.3305
```

```
followups <- pilots |>
  filter(go) |>
  mutate(planned_n = map_dbl(d_hat, n_for_80),
         actual_power = map_dbl(planned_n, \(n) pwr.t.test(n = n, d = 0.4)$power))

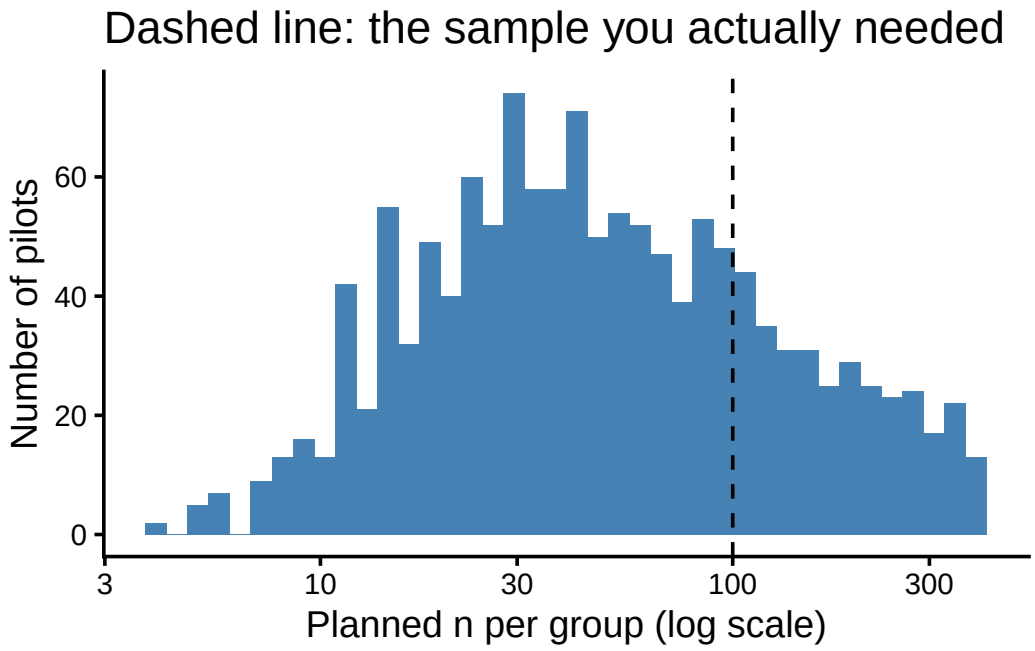
quantile(followups$planned_n, c(.1, .5, .9))
```

```
  10%   50%   90%
 14.0  44.0 198.2
```

```
mean(followups$actual_power)
```

```
[1] 0.5228514
```

```
followups |>
  ggplot(aes(x = planned_n)) +
  geom_histogram(bins = 40, fill = "steelblue") +
  scale_x_log10() +
  geom_vline(xintercept = n_for_80(0.4), linetype = "dashed") +
  labs(x = "Planned n per group (log scale)", y = "Number of pilots",
       title = "Dashed line: the sample you actually needed")
```



To do: The sample you actually needed is `n_for_80(0.4)` per group. What did your pilots tell you to run instead? Would you trust an effect size from your 10-per-group pilot?

Hew Note: Wow you need 100 people per group.

Part 5: Small telescopes, or how big should a replication be?

So we can't trust the original effect. For replications there's a neat way around that. Simonsohn (2015) changes the question from "is the effect exactly what they said?" to "is the effect too small for the original study to have seen?"

Call **d33** the effect size that the original study had 33% power to detect. If the true effect is smaller than that, the original study was a telescope too small to see it. The rule of thumb on the slides says that a replication with 2.5 times the original sample has about 80% power to rule out d33 when the true effect is zero. Let's test that rule.

Suppose the original study had 20 people per group.

To do: Use `pwr.t.test()` to find d33. You know `n` and `power`, so leave out `d`.

```
original_n <- 20
```

```
d33 <- pwr.t.test(n = original_n, power = .33, sig.level = .05)$d
d33
```

```
[1] 0.4929019
```

Now simulate replications with **no true effect**, and ask how often we can reject d33. Rejecting means the replication's estimate is reliably *below* d33, so it's a one-sided test of the difference against `d33`, not against zero. In words: rejecting d33 lets you conclude that the true effect, whatever it is, is smaller than d33. Equivalently, the replication's 90% confidence interval lies entirely below d33. `sd = 1` here, so the difference between means is already in units of d.

```
rejects_d33 <- function(n_per_group, d33, true_d = 0) {
  treatment <- rnorm(n_per_group, mean = true_d, sd = 1)
  control <- rnorm(n_per_group, mean = 0, sd = 1)
  t.test(treatment, control, mu = d33, alternative = "less", var.equal = TRUE)$p.value < .05
}

power_to_reject <- function(n_per_group, d33, true_d = 0, n_sims = 2000) {
  mean(map_lgl(1:n_sims, \(i) rejects_d33(n_per_group, d33, true_d)))
}
```

To do: What's the power to reject d33 with 2.5 × the original sample? With the same sample as the original?

```
set.seed(5)

power_to_reject(n_per_group = 20, d33 = d33)
```

```
[1] 0.4625
```

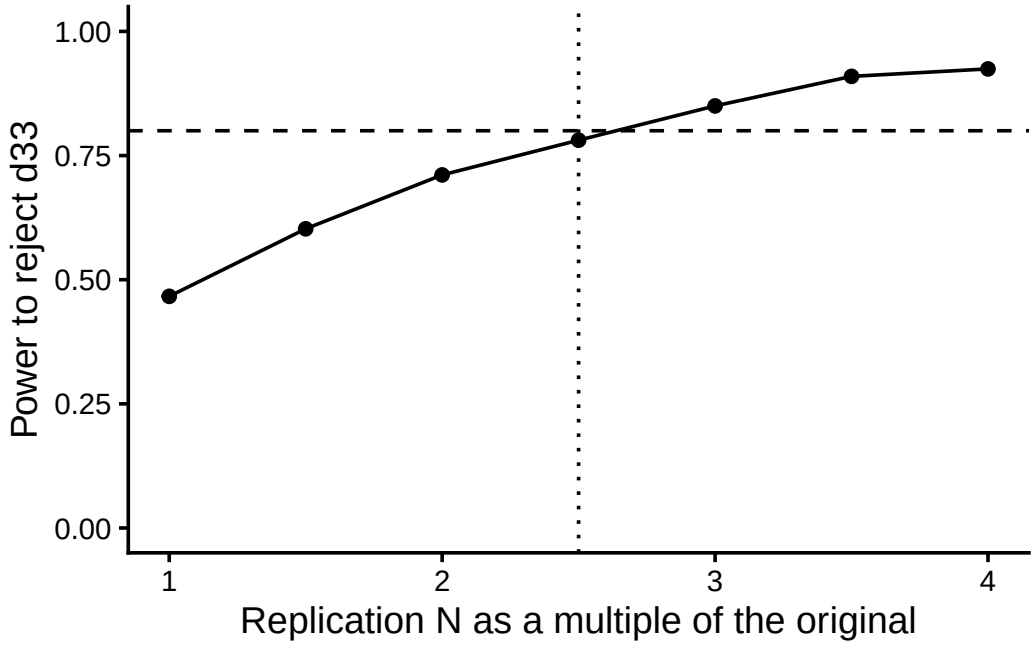
```
power_to_reject(n_per_group = 50, d33 = d33)
```

```
[1] 0.7785
```

To do: Plot power to reject d33 as replication size goes from 1× to 4× the original. Use the `telescope` table, with `multiplier` on the x-axis and `power` on the y-axis, and `geom_line()`, as for the power curves in Lab 2. Add a dashed line at .8 with `geom_hline(yintercept = .8, linetype = "dashed")`.

```
telescope <- tibble(multiplier = seq(1, 4, by = 0.5)) |>
  mutate(n_per_group = multiplier * original_n,
         power = map_dbl(n_per_group, \(n) power_to_reject(n, d33)))

telescope |>
  ggplot(aes(x = multiplier, y = power)) +
  geom_line() +
  geom_point() +
  geom_hline(yintercept = .8, linetype = "dashed") +
  geom_vline(xintercept = 2.5, linetype = "dotted") +
  ylim(0, 1) +
  labs(x = "Replication N as a multiple of the original", y = "Power to reject d33")
```



That is crazy it is also almost about 2.5.

To do: Answer these questions.

1. Does the 2.5× rule hold up? Yes, the 2.5x rule actually holds up quite well. The original study had .46 power and with a 2.5x sample size, we get the power level of .78 which is very close to the 80% power that we wanted.

I have heard of a double the sample size rule and that already made me go ok that is wild but in this case, 2.5x did not technically even reach 80% power.

2. Why is this a better plan for a replication than powering on the original effect?

The original effect may be unreliable.

3. What are the possible conclusions if your replication is *not* significant? When can you say "the effect is too small for the original study to have detected" and when can you only say "inconclusive"? (Hint: picture the replication's 90% CI. Where can it fall relative to 0 and to d33?)

Part 6: When your sample is fixed, flip the question

Most student projects are limited by resources: credits on SONA, weeks left in the quarter. When N is set by what's feasible, don't pretend the N came from a power analysis. Say what

is true: here is the N we can get, and here is what it can and can't detect. Then a reader can decide if the study is informative. This is a **sensitivity analysis**.

Suppose you can get 120 SONA participants, 60 per group.

To do: Fill in the blanks. What's the smallest effect you have 80% power to detect? What's your power across a range of effect sizes? In the `sensitivity` chunk, `\(this_d) ...` handles one effect size at a time, so use `this_d` inside `pwr.t.test()`, not `d`: `d` is the whole column of 19 effect sizes, and `pwr.t.test()` would give an error.

```
fixed_n <- 60 # per group
```

```
smallest_d <- pwr.t.test(n = fixed_n, power = .8, sig.level = .05)$___
smallest_d

sensitivity <- tibble(d = seq(0.1, 1, by = 0.05)) |>
  mutate(power = map_dbl(d, \(this_d) pwr.t.test(n = ___, d = ___)$power))
```

```
Error in parse(text = input): <text>:1:68: unexpected input
1: smallest_d <- pwr.t.test(n = fixed_n, power = .8, sig.level = .05)$_
                                                                      ^
```

To do: Plot `power` against `d`, using the `sensitivity` table and `geom_line()`. Add a dashed line at .8 with `geom_hline()`.

The other half of this argument is the **smallest effect size of interest** (SESOI): the smallest effect you would actually care about. That's a scientific decision, not a statistical one. If your smallest effect of interest is d = .3, the curve above says how likely you are to miss it, and you can decide if you're comfortable with that.

To do: Finish this justification, following the example on the slides. The numbers should come from your code, using inline R as in Lab 1. Inline R is code inside your text: in the `.qmd` it is a backtick, the letter r, a space, your code, and a closing backtick. When you render, it is replaced by its result. The paragraph below has three of these, each holding `"___"`. Replace each `"___"`, **including the quotes**, with code. Power comes out between 0 and 1, so multiply by 100 and round. For example, the first one should hold `round(pwr.t.test(n = fixed_n, d = 0.3)$power * 100)`.

> We did not have strong prior information about the likely effect size, so we were interested in effects as small as d = .3. We were limited to a maximum sample of 120 SONA participants. With this sample we have ***% power for d = .3,*** % power for d = .5, and 80% power for effects of d = _____ or larger.

To do: In the reading responses, someone asked how a study can be "overpowered." With 100,000 people per group, what's your power for d = .02? Is more data a problem? What *is* the problem?

```
pwr.t.test(n = ___, d = ___)$power
```

```
Error in parse(text = input): <text>:1:17: unexpected input
1: pwr.t.test(n = __
                    ^
```

Part 7: More people or more trials?

So far each person gave us one number. Many of your studies have many trials per person, and then "sample size" has two parts. Think back to Lab 3: averaging more ratings per person makes each person's score more reliable. But people also differ from each other, and more trials can't fix that.

Each person's average across `n_trials` trials has two sources of variability: real differences between people (`sd_person`) and trial noise, which shrinks as trials go up (`sd_trial / sqrt(n_trials)`).

```
simulate_person_means <- function(n_per_group, n_trials, true_diff,
                                  sd_person = 1, sd_trial = 4) {
  sd_of_means <- sqrt(sd_person^2 + sd_trial^2 / n_trials)
  treatment <- rnorm(n_per_group, mean = true_diff, sd = sd_of_means)
  control <- rnorm(n_per_group, mean = 0, sd = sd_of_means)
  t.test(treatment, control, var.equal = TRUE)$p.value
}

power_people_trials <- function(n_per_group, n_trials, true_diff = 0.6,
                                sd_person = 1, n_sims = 1000) {
  mean(map_dbl(1:n_sims, \(i) simulate_person_means(n_per_group, n_trials, true_diff,
                                                    sd_person = sd_person)) < .05)
}
```

Suppose your budget is 1,200 trials per group. You could run a few people for a long time, or many people for a short time.

To do: Fill in `n_trials` so that every row uses the same 1,200 trials per group, then plot power against `n_per_group`. Add `scale_x_log10()` so that the small sample sizes don't get squashed together on the left.

```r
set.seed(7)

trade_off <- tibble(n_per_group = c(5, 10, 20, 40, 60, 120, 240, 400)) |>
  mutate(n_trials = ___,
         power = map2_dbl(n_per_group, n_trials, power_people_trials))

trade_off
```

```
Error in parse(text = input): <text>:4:22: unexpected input
3: trade_off <- tibble(n_per_group = c(5, 10, 20, 40, 60, 120, 240, 400)) |>
4:   mutate(n_trials = __
                       ^
```

To do: Answer these questions.

1. Holding total trials fixed, which matters more, people or trials? Do the gains shrink at some point? What does this leave out about the real cost of each new participant?
2. Now keep `n_per_group = 20` and raise `n_trials` to 100, 1000, and 10000, for example with `map_dbl(c(100, 1000, 10000), \(k) power_people_trials(20, k))`. What does power level off at, and why can't more trials get you past it?
3. What would change if most of the noise were between trials rather than between people? Rerun `trade_off`, replacing its `power =` line with `power = map2_dbl(n_per_group, n_trials, \(n, k) power_people_trials(n, k, sd_person = 0.2))`. How does this relate to deep-sampling studies, with a few participants and thousands of trials?

Part 8: Getting an effect size

Every power analysis so far started from an effect size someone handed you. For your project you'll have to find one yourself, either in data or in the numbers a paper reports. This part covers the common cases, and then one more thing that matters as much: how uncertain that effect size is. It uses `make_tea_data()` from Part 2 and `simulate_study()` and `n_for_80()` from Part 3.

8a. From raw data: Cohen's d by hand

Cohen's d is the difference between two means divided by the standard deviation within groups. With two groups, that SD is the **pooled SD**, a weighted average of the two groups' variances:

$$\text{pooled SD} = \sqrt{[(n - 1) \cdot s^2 + (n - 1) \cdot s^2) \,/\, (n + n - 2)]}$$

The first chunk below simulates one tea experiment with `delta = 1`, a true d of .8 as in Part 2.

```
set.seed(8)

tea <- make_tea_data(n_total = 40, delta = 1)

milk <- tea |> filter(condition == "milk first") |> pull(rating)
tea_first <- tea |> filter(condition == "tea first") |> pull(rating)
n1 <- length(milk)
n2 <- length(tea_first)
```

To do: Fill in the blanks to compute d by hand.

```
pooled_sd <- sqrt(((n1 - 1) * var(milk) + (n2 - 1) * var(___)) / (n1 + n2 - 2))
d_by_hand <- (mean(milk) - mean(tea_first)) / ___
d_by_hand
```

```
Error in parse(text = input): <text>:1:59: unexpected input
1: pooled_sd <- sqrt(((n1 - 1) * var(milk) + (n2 - 1) * var(__
                                                             ^
```

In Part 3 you got d from the t statistic with $d = t \times \sqrt{(2/n)}$. That version assumes equal groups. The general version is $d = t \times \sqrt{(1/n\ + 1/n\ )}$.

To do: Compute d from the t statistic.

```
t_stat <- unname(t.test(milk, tea_first, var.equal = TRUE)$statistic)
d_from_t <- t_stat * sqrt(___)
d_from_t
```

```
Error in parse(text = input): <text>:2:28: unexpected input
1: t_stat <- unname(t.test(milk, tea_first, var.equal = TRUE)$statistic)
2: d_from_t <- t_stat * sqrt(__
                             ^
```

To do: Do the two numbers agree? Is your estimate .8? Why not?

8b. Within-subjects designs: which d?

When the same people are measured twice, there are two common versions of d, and they can be very different.

- **d_av** divides the mean difference by the average SD of the two measurements. It is on the same scale as a between-groups d.
- **d_z** divides the mean difference by the SD of the *difference scores*. This is what `pwr.t.test(type = "paired")` needs, and what you get from a paired t-test with d_z = t / √N.

The correlation r between the two measurements links them: d_z = d_av / √(2(1 − r)).

`make_paired_data()` simulates people measured before and after. Both measurements have SD 1, the true difference is `d_av`, and the two measurements correlate at `r`.

```
make_paired_data <- function(n, d_av = 0.5, r = 0.5) {
  before <- rnorm(n)
  after <- r * before + sqrt(1 - r^2) * rnorm(n) + d_av
  tibble(person = 1:n, before, after)
}
```

To do: For correlations of .2, .5, and .8, the chunk simulates a big sample, so the estimates are close to the truth. Fill in the blanks to compute d_z and the number of people a paired study needs for 80% power. `type` takes a word in quotes. In a paired design `n` is the number of people, so you can compare `n_paired` directly with the total on the last line.

```
set.seed(81)

paired_check <- tibble(r = c(0.2, 0.5, 0.8)) |>
  mutate(data = map(r, \(this_r) make_paired_data(n = 5000, d_av = 0.5, r = this_r)),
         d_av = map_dbl(data, \(df) mean(df$after - df$before) / sqrt((var(df$before) + var(
         d_z = map_dbl(data, \(df) mean(df$after - df$before) / ___),
         n_paired = map_dbl(d_z, \(d) ceiling(pwr.t.test(d = d, power = .8, type = ___)$n)))
  select(-data)

paired_check

n_for_80(0.5) * 2 # total N for d = .5 in a between-groups design
```

```
Error in parse(text = input): <text>:6:66: unexpected input
5:          d_av = map_dbl(data, \(df) mean(df$after - df$before) / sqrt((var(df$before) + va
6:          d_z = map_dbl(data, \(df) mean(df$after - df$before) / __
                                                                    ^
```

To do: Answer these questions.

1. How does d_z change as the correlation goes up? Why? (Hint: what happens to the SD of the difference scores when the two measurements move together?)
2. A paper reports "d = .5" for a within-subjects effect and doesn't say which d. Suppose it is really a d_z from measurements that correlate at .8, and you plug it into a power analysis for a *between-groups* replication. What goes wrong?

8c. From the numbers a paper reports

Most papers won't give you their data. These are the conversions you'll need most often.

What the paper reports	How to get an effect size
Two-group t-test: t, n , n	d = t × √(1/n + 1/n)
Paired t-test: t, N	d_z = t / √N
F(1, df) comparing two groups	t = √F, then as above
Means and SDs	d = (M − M) / pooled SD, as in 8a
Means and SEMs	SD = SEM × √n, then as above
Correlation r	r is already an effect size: use `pwr.r.test(r = r, power = .8)`, where `n` is the total number of people. If you need d: d = 2r / √(1 − r²)
One-way ANOVA, F(df , df)	partial ², the share of variance explained by condition, is F · df / (F · df + df). Then Cohen's f = √(² / (1 − ²)), and use `pwr.anova.test(k = number of groups, f = f, power = .8)`, where `n` is per group

If a paper only shows a figure, you can read the means and error bars off it with WebPlotDigitizer.

Here are five made-up results sections.

1. "Participants in the warm condition (n = 30) rated the confederate as friendlier than those in the cold condition (n = 30), t(58) = 2.40, p = .02."
2. "There was a main effect of condition, F(1, 78) = 6.25, p = .01." Two groups of 40.
3. "The trained group (M = 5.2, SD = 1.1, n = 25) outperformed controls (M = 4.6, SEM = 0.24, n = 25)."
4. "Vocabulary size was correlated with looking time, r = .25, N = 100."
5. "Reaction times differed across the three conditions, F(2, 87) = 4.10, p = .02." Three groups of 30.

To do: The first one is done for you. Compute the effect size for each of the others, and the sample size for 80% power. `n_for_80()` only works for a two-group t-test, so for 4 and 5 use the `pwr` function from the table.

```
# 1. Two-group t-test
d1 <- 2.40 * sqrt(1 / 30 + 1 / 30)
d1
```

```
[1] 0.6196773
```

```
n_for_80(d1) # per group
```

```
[1] 42
```

```
# your code here
```

A quick habit worth having: check that the numbers in a paper agree with each other. A two-sided p-value comes from t with `2 * pt(-abs(t), df)`.

To do: Does the p-value in result 1 match its t?

```
2 * pt(-abs(___), df = ___)
```

```
Error in parse(text = input): <text>:1:14: unexpected input
1: 2 * pt(-abs(__
                ^
```

8d. How sure are you about that effect size?

A **95% confidence interval** (CI) is a range of effect sizes that fit the data: if you ran the same study many times, 95% of the intervals made this way would contain the true effect. An effect size from one study has a CI too, and with small samples it is wide. For two groups, the standard error of d is approximately

$$\mathrm{SE}(d) = \sqrt{[(n_1 + n_2) / (n_1 \cdot n_2) + d^2 / (2(n_1 + n_2))]}$$

```
d_ci <- function(d, n1, n2, level = .95) {
  se <- sqrt((n1 + n2) / (n1 * n2) + d^2 / (2 * (n1 + n2)))
  z <- qnorm(1 - (1 - level) / 2)
  c(lower = d - z * se, upper = d + z * se)
}
```

To do: A paper reports d = .8 with 20 people per group, like a typical published study in Part 3. What is the 95% CI? Then check the formula by simulation: if the true effect were .8, how much would 2,000 studies of that size vary? The middle 95% of the simulated `d_hat` values runs from the 2.5th to the 97.5th percentile, which `quantile()` writes as `c(.025, .975)`.

```
set.seed(82)

d_ci(d = ___, n1 = ___, n2 = ___)

sims <- tibble(study = 1:2000) |>
  mutate(result = map(study, \(i) simulate_study(n_per_group = 20, true_d = 0.8))) |>
  unnest(result)

quantile(sims$d_hat, ___) # the middle 95%
```

```
Error in parse(text = input): <text>:3:11: unexpected input
2:
3: d_ci(d = __
             ^
```

One way to use this is **safeguard power** (Perugini et al., 2014): instead of planning for the point estimate, plan for the lower end of a 60% CI around it.

To do: Find the safeguard effect size and the sample size it asks for.

```
safeguard_d <- unname(d_ci(0.8, 20, 20, level = ___)["lower"])
safeguard_d

n_for_80(0.8) # planning on the published effect
n_for_80(___) # safeguard power
```

```
Error in parse(text = input): <text>:1:50: unexpected input
1: safeguard_d <- unname(d_ci(0.8, 20, 20, level = __
                                                    ^
```

To do: Answer these questions.

1. How wide is the 95% CI, compared with the effect itself? Does the simulation agree with the formula?
2. How many more people does safeguard power ask for?
3. In Part 3, the true effect behind a published d of .8 was only .3. Does safeguard power fix that problem?

Part 9: Your project

This is what you turn in. The goal is a **Power Analysis** section you can paste into your report and your preregistration. It should make an honest case for your N, and it doesn't have to be a classic power analysis at all.

1. Name your key test. Which single test most directly captures the claim you're replicating or simulating? Write it here.

Your answer:

2. Get an effect size. If the paper doesn't report one, compute it with the conversions in Part 8c. Say which conversion you used, and for a within-subjects design, whether your effect is a d_z or a d_av (Part 8b).

Example: a paper reports a paired $t(44) = 3.98$. Then N = 45, and d_z = 3.98 / √45 = .59. For a paired t-test, `pwr` needs `type = "paired"`, and `n` is the number of people.

```
pwr.t.test(d = 3.98 / sqrt(45), power = .8, type = "paired")$n
```

```
[1] 24.28802
```

Some designs Part 8c didn't cover:

Your key test	Effect size and `pwr` function
Two proportions (e.g., % correct in two groups)	Cohen's h: `pwr.2p.test(h = ES.h(p1, p2), power = .8)`, where `n` is per group
Chi-square test of a contingency table	Cohen's w: `pwr.chisq.test(w = w, df = df, power = .8)`, where `N` is the total
Interaction in a 2×2, or a coefficient from a regression or mixed model	No simple conversion you can trust. Simulate (step 4 has a 2×2 example; the Bonus covers mixed models), and plan for an effect you can justify, not the reported one

3. Decide what you'll plan for. Don't stop at the published effect, for the reasons in Part 3. Compute at least two of these:

- Power for **half** the original effect, a simple correction for inflation (Camerer et al., 2018, on the slides, planned for 75% and then 50% of the original effect)
- The **2.5× original N** rule (Part 5)
- **Safeguard power**: the lower end of a 60% CI on the original effect (Part 8d)
- The **sensitivity** of the N you can actually get (Part 6)

The 2.5× rule is for replications: it gives you an N directly (2.5 × the original N, counting people, not groups), so report it next to the others. `d_ci()` from Part 8d only works for two groups. For a within-subjects d_z from N people, use `dz_ci()` below, and remember `type = "paired"` in `pwr.t.test()`. It is approximate, and a little too narrow for small N.

```
dz_ci <- function(dz, n, level = .95) {
  se <- sqrt(1 / n + dz^2 / (2 * n))
  z <- qnorm(1 - (1 - level) / 2)
  c(lower = dz - z * se, upper = dz + z * se)
}

dz_ci(3.98 / sqrt(45), n = 45, level = .60) # safeguard d_z for the example above
```

```
    lower     upper
0.4572482 0.7293585
```

```
# your code here
```

4. Make a power curve. Pick the option that fits your key test.

Your key test	What to use
t-test or correlation	`pwr.t.test()` or `pwr.r.test()`, as in Parts 1 and 6
One-way ANOVA	`pwr.anova.test()`, or simulate
Interaction, mixed model, nonparametric test, anything else	Simulate. Start from the examples and the wrapper below, or your Lab 2 or project simulation
Simulation project	Simulate. Decide first what counts as recovering the key effect (e.g., the key parameter is significant in the predicted direction, or its estimate lands within some range of the true value), then ask how often that happens as N, or the number of runs, goes up

To simulate, write a function that makes one fake dataset in the shape of *your* study, runs *your* planned test, and returns its p-value. Then the rest stays the same.

Here are two finished examples to copy from. The first is a two-group t-test, so you can check it against `pwr`. Its `n` is per group, and `effect` is Cohen's d.

```r
example_experiment <- function(n, effect) {
  treatment <- rnorm(n, mean = effect, sd = 1)
  control <- rnorm(n, mean = 0, sd = 1)
  t.test(treatment, control, var.equal = TRUE)$p.value
}

set.seed(91)
mean(map_dbl(1:500, \(i) example_experiment(64, effect = 0.5)) < .05) # simulation
```

```
[1] 0.826
```

```r
pwr.t.test(n = 64, d = 0.5)$power # formula
```

```
[1] 0.8014596
```

```r
mean(map_dbl(1:500, \(i) example_experiment(64, effect = 0)) < .05) # should be near .05
```

```
[1] 0.038
```

The second is the interaction in a between-subjects 2×2. Here `n` is per cell, so the total is `4 * n`, and `effect` is the difference between the two simple effects, in SD units. Coding each factor as −0.5 and 0.5 makes the `a:b` coefficient equal to exactly that difference.

```r
interaction_experiment <- function(n, effect) {
  cells <- expand_grid(a = c(-0.5, 0.5), b = c(-0.5, 0.5), person = 1:n) |>
    mutate(y = effect * a * b + rnorm(n(), mean = 0, sd = 1))
  summary(lm(y ~ a * b, data = cells))$coefficients["a:b", "Pr(>|t|)"]
}

set.seed(92)
mean(map_dbl(1:500, \(i) interaction_experiment(64, effect = 0.5)) < .05)
```

```
[1] 0.512
```

With 64 per cell, 256 people in total and twice as many as the t-test above, power for an interaction of .5 is only about 50%. Interactions need big samples. If your study is within-subjects, simulate one row per trial instead, as in the Bonus.

Now write your own. The chunk below has `#| eval: false` so that the lab still renders before you've written anything. **Delete that line once your `one_experiment()` works**, or your power curve won't be in your rendered file.

```r
one_experiment <- function(n, effect) {
  # 1. simulate one dataset with n participants (say whether n is the total
  #    or per group) and a true effect of size effect
  # 2. run your confirmatory test on it
  # 3. return the p-value
}

my_power_curve <- tibble(n = seq(20, 200, by = 20)) |>
  mutate(power = map_dbl(n, \(this_n) mean(map_dbl(1:200, \(i) one_experiment(this_n, effect

ggplot(my_power_curve, aes(x = n, y = power)) +
  geom_line() +
  geom_hline(yintercept = .8, linetype = "dashed") +
  ylim(0, 1)
```

Start with `1:200` while you debug. Use more once it works. Check that your function gives power near .05 when you run it with `effect = 0`: if it doesn't, something is wrong with your simulation or your test.

5. Write the justification. One paragraph with your numbers, in the style of Part 6: your goal, your constraint, your planned N, and what that N can and can't detect. This goes in the Power Analysis section of your report, and it's a draft of the sampling plan for your preregistration.

Your numbers from step 3 will probably disagree, and the most cautious one may be more people than you can get. That's normal. You don't have to run the largest N. Say what N you will actually run and why, then say what that N is powered for under each approach. If it falls short of what your planned effect needs, say so plainly and lean on the sensitivity analysis. "This is the N we could get, and here is what it can detect" is an honest justification (Lakens, 2022, calls it a resource constraint). What isn't honest is choosing an effect size after the fact so that your N looks powered.

Here is an example for a replication, where the original study had 20 people per group and reported d = .8:

> The original study (N = 40) reported d = .80. Because published effects tend to be inflated, we planned for half that effect, d = .40, which needs 100 participants per group for 80% power. That is more than we can recruit this quarter, so we will run 60 per group (N = 120). This is 3 times the original sample, which gives more than 80% power to reject d33 = .50, the effect the original study had 33% power to detect (Simonsohn, 2015). It also gives 80% power for effects of d = .52 or larger. A non-significant result would tell us whether the effect is as large as the original study could have detected, but not whether there is a smaller effect, below about d = .5.

Your paragraph:

To submit: commit and push, then submit your repository link on Canvas with your power curve figure and your paragraph. To save the figure as a file, run `ggsave("power_curve.png")` in the Console right after making the plot, or right-click the figure in your rendered HTML and save it.

Bonus: mixed models

If your key test is a mixed model, your simulated dataset needs one row per trial, and `one_experiment()` should fit the model. Here is a worked example for a within-subjects effect with a random intercept and slope for each person. It needs two more packages, `lme4` and `lmerTest`, and it is slow, so try it with few simulations first. If you see `singular` warnings at small N, that's common: your model can't estimate the random effects well from so few people.

```
install.packages(c("lme4", "lmerTest")) # run once
```

```
library(lmerTest)

one_lmm_experiment <- function(n_people, n_trials = 20, effect = 0.2,
                               sd_intercept = 0.5, sd_slope = 0.2, sd_trial = 1) {
  people <- tibble(person = 1:n_people,
                   intercept = rnorm(n_people, 0, sd_intercept),
                   slope = rnorm(n_people, effect, sd_slope))
  trials <- people |>
    cross_join(tibble(trial = 1:n_trials)) |>
    mutate(condition = if_else(trial %% 2 == 0, 0.5, -0.5), # half the trials in each conditi
           y = intercept + slope * condition + rnorm(n(), 0, sd_trial))
  fit <- suppressMessages(lmer(y ~ condition + (1 + condition | person), data = trials))
  summary(fit)$coefficients["condition", "Pr(>|t|)"]
}

set.seed(9)

lmm_power <- tibble(n_people = c(20, 40, 80)) |>
  mutate(power = map_dbl(n_people, \(n) mean(map_dbl(1:100, \(i) one_lmm_experiment(n)) < .05

lmm_power
```

Doing the same thing with `simr`

Writing `one_lmm_experiment()` yourself shows you exactly what a power simulation does. The `simr` package (Green & MacLeod, 2016) does the same job with less code. You give it a fitted model, and it simulates new datasets from that model, refits the model to each one, and counts how often the effect is significant.

It works in three steps:

1. **Fit your model once** to some data: your pilot data, or one fake dataset like the one below.
2. **Set the effect to the value you want to plan for**, with `fixef()`. Use a value you can justify, not whatever your pilot happened to show (Part 4).
3. **Ask for power.** `powerSim()` gives power at the sample size you have. `extend()` and `powerCurve()` give power at larger sample sizes.

`simr` also borrows the random-effect SDs and the trial-level SD from the model you fit in step 1. With a small pilot those are uncertain too, so treat the answer as a rough guide.

```
install.packages("simr") # run once
```

```
library(lmerTest)
library(simr)

set.seed(10)

# Step 1: one fake dataset with 20 people and 20 trials each, the same design
# as one_lmm_experiment() above, and the model fit to it
pilot_people <- tibble(person = 1:20,
                       intercept = rnorm(20, 0, 0.5),
                       slope = rnorm(20, 0.2, 0.2))
pilot <- pilot_people |>
  cross_join(tibble(trial = 1:20)) |>
  mutate(condition = if_else(trial %% 2 == 0, 0.5, -0.5),
         y = intercept + slope * condition + rnorm(n(), 0, 1))

pilot_fit <- lmer(y ~ condition + (1 + condition | person), data = pilot)
fixef(pilot_fit) # the effect this fake pilot happened to give

# Step 2: plan for the effect you can justify
fixef(pilot_fit)["condition"] <- 0.2

# Step 3a: power with the 20 people you have
```

```r
powerSim(pilot_fit, test = fixed("condition", "t"), nsim = 100, progress = FALSE)

# Step 3b: power with 20, 40, and 80 people
bigger_fit <- extend(pilot_fit, along = "person", n = 80)
simr_curve <- powerCurve(bigger_fit, test = fixed("condition", "t"), along = "person",
                         breaks = c(20, 40, 80), nsim = 100, progress = FALSE)
simr_curve
plot(simr_curve)
```

To use this for your own study, change three things: the data in step 1 (your pilot, or a fake dataset shaped like your study), the formula in `lmer()`, and the name of your effect, here `"condition"`, everywhere it appears. `nsim = 100` takes about 20 seconds here. Use more, say 500, for the numbers you report. If the Console fills with `boundary (singular) fit` messages, that's the same warning described above, and you can ignore it here.

To do: Compare `simr`'s power at 20, 40, and 80 people with `lmm_power` from the chunk above. Should they agree?

References

- Albers, C., & Lakens, D. (2018). When power analyses based on pilot data are biased: Inaccurate effect size estimators and follow-up bias. *Journal of Experimental Social Psychology, 74*, 187–195.
- Green, P., & MacLeod, C. J. (2016). SIMR: An R package for power analysis of generalized linear mixed models by simulation. *Methods in Ecology and Evolution, 7*(4), 493–498.
- Lakens, D. (2013). Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for t-tests and ANOVAs. *Frontiers in Psychology, 4*, 863.
- Lakens, D. (2022). Sample size justification. *Collabra: Psychology, 8*(1), 33267.
- Perugini, M., Gallucci, M., & Costantini, G. (2014). Safeguard power as a protection against imprecise power estimates. *Perspectives on Psychological Science, 9*(3), 319–332.
- Simonsohn, U. (2013). [4] The folly of powering replications based on observed effect size. Data Colada.
- Simonsohn, U. (2015). Small telescopes: Detectability and the evaluation of replication results. *Psychological Science, 26*(5), 559–569.